Week 7 — Multi-Agent Reinforcement Learning

Notebook Companion Guide

From Conway to LangGraph: Agent Systems for Physicists in the LLM Era
Università di Bologna · Dipartimento di Fisica

Section	Topic	Time
0	Installation & imports	5 min
1	PettingZoo — the multi-agent API	10 min
2	Nash Equilibrium: Q-learners on RPS	25 min
3	Grim Trigger: cooperation through credible threats	20 min
4	Cooperative Navigation: environment exploration	15 min
5	Independent PPO on Cooperative Navigation	30 min
6	MADDPG: Centralised Training, Decentralised Execution	20 min
7	Bridge to Generative ABM	10 min

1. Section 0 — Installation & Setup

1.1 What gets installed

```
pip install pettingzoo[classic,mpe] stable-baselines3 supersuit gymnasium
    matplotlib numpy
```

The `[classic]` extra provides card and board-game environments (RPS, Blackjack, Chess). The `[mpe]` extra provides the Multi-Particle Environments used in Sections 4–6 (Cooperative Navigation, Predator-Prey). SuperSuit is the adapter library that wraps PettingZoo environments in a format Stable-Baselines3 can consume.

Installation time on Colab. Expect 90–120 seconds. MuJoCo and Atari extras are *not* installed—both require system libraries that may fail on Colab's runtime image. All cells in this notebook use `continuous_actions=False`, so Box2D and MuJoCo are not needed.

2. Section 1 — PettingZoo: The Multi-Agent API

2.1 Why a new API?

Gymnasium assumes one agent and one environment. In multi-agent settings, two structural questions immediately arise:

1. **Turn order.** Do agents act simultaneously or sequentially?
2. **Heterogeneity.** Can agents have different observation/action spaces?

PettingZoo answers both with the **Agent-Environment Cycle (AEC)** model: agents act one at a time in a fixed cycle, and each agent has its own spaces.

2.2 AEC vs Parallel API

PettingZoo offers two interfaces:

API	Call signature	Use when
AEC	`env.step(action)` per agent	Turn-based games, board games
Parallel	`env.step(action_dict)`	Simultaneous action (physics sims)

We use the Parallel API for training (Sections 4–5) because the MPE environments model continuous-time dynamics where all agents act at the same clock tick.

2.3 The AEC loop

```
from pettingzoo.classic import rps_v2
env = rps_v2.env()
env.reset()

for agent in env.agent_iter():
    obs, reward, terminated, truncated, info = env.last()
    if terminated or truncated:
        env.step(None)          # dead agents must pass None
    else:
        action = policy[agent](obs)
        env.step(action)
```

Physics note: self-referential potential. In Gymnasium one agent lives in a fixed potential $V(s)$. In PettingZoo each agent i lives in a potential $V_i(s; \pi_{-i})$ that depends on all other agents' policies. When the other agents update, the potential changes— this is the *non-stationarity* problem that makes convergence guarantees so much harder to establish. Single-agent RL is one particle in a fixed field; multi-agent RL is N particles each generating the field for the others.

3. Section 2 — Nash Equilibrium: Q-Learners on Rock-Paper-Scissors

3.1 Nash equilibrium as a fixed point

A Nash equilibrium $(\pi_1^*, \ldots, \pi_N^*)$ satisfies

$$V_i(\pi_i^*, \pi_{-i}^*) \;\geq\; V_i(\pi_i, \pi_{-i}^*) \quad \forall\, \pi_i,\ \forall\, i. \tag{1}$$

This is the multi-agent counterpart of the Bellman fixed point Q^*: just as Q^* is the unique fixed point of the Bellman operator $\mathcal{T}$, the Nash equilibrium is the fixed point of the *joint best-response operator*— no agent can improve by deviating unilaterally.

Nash's existence theorem (1950). Every finite game has at least one Nash equilibrium, possibly in mixed strategies (probability distributions over actions). A pure-strategy NE does not always exist—RPS is the canonical counterexample.

3.2 RPS and maximum entropy

The unique Nash equilibrium of symmetric RPS is the mixed strategy $\pi^*(R) = \pi^*(P) = \pi^*(S) = 1/3$. This is the **maximum-entropy distribution** over three outcomes:

$$H(\pi^*) = -\sum_a \pi^*(a) \log \pi^*(a) = \log |\mathcal{A}| = \log 3 \approx 1.099 \text{ nats}. \tag{2}$$

Nash = Boltzmann at $T \to \infty$. Any bias in a player's strategy is exploitable. The optimal response removes all exploitable structure—it maximises entropy, placing all actions at equal probability. This is precisely the Boltzmann distribution $p(a) \propto e^{-E(a)/T}$ in the

limit $T \to \infty$ (infinite temperature, all states equally likely). Rational play under perfect mutual information forces maximum disorder.

3.3 Two independent Q-learners

Each agent maintains a Q-vector of length 3 (one per action) and uses softmax exploration:

$$\pi(a \mid Q, \tau) = \frac{e^{Q(a)/\tau}}{\sum_{a'} e^{Q(a')/\tau}}. \tag{3}$$

The temperature τ is annealed from $\tau_0 = 2.0$ to $\tau_f = 0.05$.

What to observe in the plots:

- **Left panel (entropy convergence).** Both agents' policy entropies approach $\log 3 \approx 1.099$ nats, confirming that independent Q-learning finds the Nash equilibrium.
- **Right panel (Q-value oscillation).** Despite entropy convergence, the Q-values oscillate throughout training and never converge to a fixed point. This is *non-stationarity made visible*: the potential each agent optimises is continuously changing as the other agent updates.

Key distinction. The *policy* converges (entropy $\to \log 3$); the *Q-values* do not (persistent oscillation). In single-agent RL, policy convergence implies Q-value convergence. In multi-agent RL, this implication breaks down because the Q-function depends on the other agents' (time-varying) policies.

3.4 Minimax Q-learning (zero-sum case)

For 2-player zero-sum games the Bellman operator has a minimax form:

$$Q(s, a_0, a_1) = R + \gamma \cdot \max_{a_0'} \min_{a_1'} Q(s', a_0', a_1'). \tag{4}$$

This operator is a contraction and minimax Q-learning converges to the Nash equilibrium. The key insight: agent 0 maximises assuming agent 1 will minimise— both agents are modelled as rational adversaries.

4. Section 3 — Grim Trigger: Cooperation Through Credible Threats

4.1 The Prisoner's Dilemma

The Prisoner's Dilemma payoff matrix with $T > R > P > S$:

	Agent 1: Cooperate	**Agent 1: Defect**
Agent 0: Cooperate	R, R $(3, 3)$	S, T $(0, 5)$
Agent 0: Defect	T, S $(5, 0)$	P, P $(1, 1)$

One-shot Nash: mutual defection $(P, P) = (1, 1)$—individually rational but collectively suboptimal. The Pareto optimum $(R, R) = (3, 3)$ requires coordination.

4.2 Grim Trigger strategy

Cooperate until the opponent defects. If the opponent ever defects, defect forever.

4.3 Stability condition

Grim Trigger is a Nash equilibrium of the infinite repeated game iff defection is not profitable. A one-shot defection gives immediate gain $T - R$ but triggers permanent punishment. The defection is not profitable iff the immediate gain is less than the discounted future loss:

$$T - R \leq \frac{\gamma}{1-\gamma}(R - P) \iff \boxed{\gamma \geq \gamma^* = \frac{T-R}{T-P}.} \tag{5}$$

With $T = 5, R = 3, P = 1$: $\gamma^* = \frac{5-3}{5-1} = 0.5$.

The Q-learning connection. This is *identical* to the recursive inequality that arises in Q-learning when comparing immediate reward vs discounted future value. The discount factor γ appears in both places as the weight on the future. In Q-learning, $\gamma < 1$ is required for the Bellman operator to be a contraction; in Grim Trigger, $\gamma \geq \gamma^*$ is required for cooperation to be stable. The discount factor is simultaneously a convergence parameter and a cooperation parameter.

Phase transition analogy. Cooperation is the *ordered phase*; defection is the *disordered phase*. The discount factor γ plays the role of inverse temperature $\beta = 1/T$. At $\gamma < \gamma^*$ (high temperature) the system is disordered; at $\gamma \geq \gamma^*$ it orders into the cooperative ground state. The condition $\gamma = \gamma^*$ is the phase boundary—a cooperation–defection phase transition.

4.4 Strategy comparison

Matchup	Avg reward	Notes
Grim vs Grim	$\approx R = 3.0$	Sustained cooperation
Tit-for-Tat vs Tit-for-Tat	$\approx R = 3.0$	Cooperative + forgiving
AllD vs AllD	$P = 1.0$	Nash defection trap
Grim vs AllD	S then P	Defector exploits once, then mutual punishment

The right panel of the Grim Trigger figure shows the phase diagram: green region ($\gamma \geq 0.5$) sustains cooperation; red region ($\gamma < 0.5$) collapses to defection.

5. Section 4 — Cooperative Navigation: Environment

5.1 Environment description

`simple_spread_v3` is the canonical cooperative MARL benchmark. N agents must cover N landmarks while avoiding collisions.

Property	Value
Agents	3 (configurable)
Landmarks	3 (= number of agents)
Observation	18-dim vector (own vel, own pos, landmark displacements, other agents)
Action	5 discrete: no-op, left, right, up, down
Reward	Shared: $-\sum_k \min_i d(i,k)$ minus collision penalty
Episode length	25 steps

5.2 Observation breakdown (N=3)

```
obs[0:2]     own velocity (vx, vy)
obs[2:4]     own position (x, y)
obs[4:10]    displacements to 3 landmarks: (dx, dy) x 3
obs[10:18]   relative positions of 2 other agents: (dx, dy) x 2
Total: 2 + 2 + 6 + 4 = 14  (note: exact dim depends on PettingZoo version)
```

5.3 Why this environment?

Spontaneous symmetry breaking. The shared reward creates a joint free energy $\mathcal{F}(\pi_1, \ldots, \pi_N)$ over agent policies. The global optimum requires each agent to cover a *different* landmark—breaking the permutation symmetry of the initialisation. Without coordination, the system gets stuck in a symmetric saddle point: all agents crowd the same landmark. This is exactly the symmetry-breaking mechanism in ordered phases, where the ground state has lower symmetry than the Hamiltonian.

5.4 Random baseline

The random policy cell (Section 4.2) establishes a floor for episode return. Because the reward is negative distance (target: 0), larger (less negative) values are better. Expected random policy return: approximately -35 to -45 per episode. A well-trained IPPO should reach -20 to -30, and a well-trained MADDPG should reach -15 to -25.

6. Section 5 — Independent PPO

6.1 The IPPO approach

Independent PPO (IPPO) treats each agent as an independent single-agent PPO learner, ignoring the other agents entirely.

Property	IPPO
Policy	Each agent has its own MLP
Critic	Sees only agent's own observation
Assumption	Stationarity (violated in MARL)
Convergence	No guarantee—but often works
Scalability	$O(N)$ parameters, $O(1)$ inference

6.2 SuperSuit wrapping

SB3 expects a Gymnasium-style `VecEnv`. SuperSuit converts the PettingZoo Parallel env in two steps:

```
import supersuit as ss

env = simple_spread_v3.parallel_env(N=3, ...)
env = ss.pad_observations_v0(env)          # make all obs same shape
env = ss.pettingzoo_env_to_vec_env_v1(env)  # each agent is a VecEnv slot
env = ss.concat_vec_envs_v1(env, 4, num_cpus=1,
    base_class='stable_baselines3')
```

The result is a `VecEnv` with $N_{\text{agents}} \times N_{\text{envs}} = 3 \times 4 = 12$ slots. All agents share the *same network weights*—this is **parameter sharing**, which acts as a form of implicit communication: agents trained together develop a shared language of behaviours.

6.3 Training hyperparameters

Hyperparameter	Value	Rationale
Total timesteps	300 000	∼5 min on Colab CPU
Learning rate	3×10^{-4}	Standard Adam default
`n_steps`	128	Short rollouts for fast feedback
`batch_size`	256	Covers several full episodes
`ent_coef`	0.01	Prevent premature convergence
γ	0.99	Standard; cooperation requires γ close to 1

6.4 Why does IPPO work at all?

Despite non-stationarity, IPPO succeeds because:

1. **Shared reward.** All agents receive the same gradient signal direction.
2. **Parameter sharing.** Agents are initialised identically but experience different random samples, driving emergent specialisation.
3. **Slow non-stationarity.** PPO updates are small (trust region); from one agent's perspective, the others change slowly enough to approximate stationarity.

Expected results (300k steps). Mean episode return: from ~ -40 (random) to ~ -25 (IPPO). Full convergence requires ∼1M steps. The training curve will show high variance—characteristic of the non-stationarity that IPPO does not explicitly handle.

7. Section 6 — MADDPG: Centralised Training, Decentralised Execution

7.1 The CTDE paradigm

IPPO's critical flaw: each agent's critic sees only its own observation, so the environment appears non-stationary. **MADDPG** (Lowe et al., 2017) breaks the symmetry between training and execution:

Phase	Information available	Used by
Training	$(\mathbf{o}, \mathbf{a}) = (o_1, \ldots, o_N, a_1, \ldots, a_N)$	Centralised critic $Q_i(\mathbf{o}, \mathbf{a})$
Execution	o_i only (local)	Decentralised actor $\mu_i(o_i)$

7.2 Policy gradient

The MADDPG policy gradient for agent i:

$$\nabla_{\theta_i} J = \mathbb{E}[\nabla_{\theta_i} \log \pi_{\theta_i}(a_i \mid o_i) \cdot Q_i^{\pi}(\mathbf{o}, \mathbf{a})] \,. \tag{6}$$

The critic $Q_i^{\pi}(\mathbf{o}, \mathbf{a})$ uses the *joint* observation-action pair, making the environment appear stationary to the gradient estimator even though the individual agents are moving.

7.3 Architecture

The centralised critic input grows as $N \times (d_{\text{obs}} + d_{\text{act}})$:

```
class CentralisedCritic(nn.Module):
    def __init__(self, n_agents, obs_dim, act_dim, hidden=64):
        super().__init__()
        in_dim = n_agents * (obs_dim + act_dim)   # all obs + all actions
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden), nn.ReLU(),
```

```
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1),
        )
    def forward(self, all_obs, all_acts):
        x = torch.cat([all_obs, all_acts], dim=-1)
        x = x.view(x.shape[0], -1)
        return self.net(x)
```

CTDE ≡ mean-field theory. The centralised critic is the mean-field approximation operationalised: at train time, every agent "sees" the average field generated by all others (the joint critic provides global information). At test time, each agent runs locally under the frozen field. Mean-field theory is exact when correlations are negligible; CTDE is exact when the critic can be perfectly conditioned on the joint state. Both approximate the same ideal: replace many-body interactions with a single effective field.

7.4 IPPO vs MADDPG comparison

Property	IPPO	MADDPG
Critic input	Local o_i	Joint $(\mathbf{o}, \mathbf{a})$
Actor input	Local o_i	Local o_i
Non-stationarity	Visible (problem)	Hidden by joint critic
Training complexity	$O(N)$	$O(N^2)$ critic input
Physics analogy	N decoupled particles	Mean-field theory
Convergence guarantee	None	None (but more stable)
Scalability ($N \gg 3$)	Good	Poor

8. Section 7 — Bridge to Generative ABM

8.1 Limits of policy-gradient agents

IPPO and MADDPG agents are powerful but brittle in a specific way: their behaviour is fully determined by a fixed neural network trained on a specific reward signal in a specific environment. They cannot reason in natural language, plan without reward signals, communicate rich intentional content, or generalise to novel tasks without retraining.

8.2 The generative ABM paradigm

In a **Generative ABM**, agents are not policies—they are language model instances with memory and goals:

Property	MARL agent	Generative ABM agent
State	Observation vector	Memory stream (text)
Policy	$\text{MLP}(o) \to a$	$\text{LLM}(\text{context, memory}) \to \text{text} \to a$
Reward	Scalar signal	None (or implicit via reflection)
Learning	Gradient descent	In-context, no weight update

8.3 Synthetic populations

A **synthetic population** is a set of LLM agents each conditioned on a demographic-statistical profile drawn from survey data. The LLM role-plays the corresponding individual as a statistical representative of a real population segment.

Ensemble interpretation. A synthetic population is a microstate ensemble: each agent is a degree of freedom drawn from the empirical distribution $P(\text{demographics})$. The macroscopic observable (e.g., policy approval rate) is the ensemble average over all agent responses—the thermal average in the corresponding statistical-mechanics language. Demographic heterogeneity is the disorder; the survey statistic is the order parameter.

8.4 Key references

- Park, J.S. et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior.* arXiv:2304.03442.
- Argyle, L.P. et al. (2023). *Out of One, Many: Using Language Models to Simulate Human Samples. Political Analysis.* arXiv:2209.06899.
- Lowe, R. et al. (2017). *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments.* NeurIPS. arXiv:1706.02275.
- Nash, J. (1950). Equilibrium points in n-person games. *Proceedings of the National Academy of Sciences*, 36(1), 48–49.

9. Exercises

Exercise 1 — Nash entropy (25 min) Modify the RPS Q-learner to use ε-greedy instead of softmax exploration. Does the policy still converge to the Nash entropy $\log 3$? Plot entropy trajectories for both exploration schemes. *Concept tested: exploration strategy, Nash convergence, entropy.*

Exercise 2 — Phase transition (30 min) Verify the Grim Trigger phase boundary numerically. Simulate the iterated PD for $\gamma \in \{0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$ where both agents play Grim Trigger but with probability $p = 0.05$ of accidental defection. Plot mean cooperation rate vs γ. Where is the transition? *Concept tested: discount factor, phase transition, noise robustness.*

Exercise 3 $\star$ — Critic input ablation (45 min) Implement a toy centralised critic for 2 agents in a 1D environment. Compare: (A) critic sees joint observations (o_1, o_2) only; (B) critic sees (o_1, o_2, a_1, a_2). Which gives lower-variance gradient estimates? Measure empirically. *Concept tested: MADDPG design, variance reduction, gradient estimation.*

Exercise 4 $\star\star$ — GABM opinion dynamics (60 min) Use the synthetic population from Section 7. Implement the Deffuant bounded-confidence model: two agents interact if $|\theta_i - \theta_j| < \varepsilon$, updating beliefs toward each other. Vary $\varepsilon \in \{0.3, 0.5, 0.7, 1.0\}$ and plot the opinion cluster structure. Compare with the Ising/Potts mean-field prediction. *Concept tested: opinion dynamics, bounded confidence, phase transitions in social systems.*